

```
#!/usr/bin/env python3
"""
ASHE COMPLETE SYSTEM EXTRACTOR
=====

This single file contains the entire Ashe system.
Run this script to extract all files and create the complete directory
structure.

Usage: python ASHE_COMPLETE_EXTRACTOR.py

Author: Rickey Jay Bennett II
Version: 1.0.0
"""

import os
import sys

def create_directory_structure():
    """Create all necessary directories."""
    dirs = [
        'core',
        'perception',
        'web_access',
        'setup',
        'config',
        'docs',
        'packaging'
    ]
    for d in dirs:
        os.makedirs(d, exist_ok=True)
    print("✓ Directory structure created")

def extract_files():
    """Extract all embedded files."""

    files = {
        'core/__init__.py': ''''''

```

Ashe Core Module

=====

Core functionality for Ashe (Adversarial Sentient Heuristic Engine).

Components:

- ashe_chat: Interactive chat interface
- ashe_daemon: Autonomous thinking daemon

Author: Rickey Jay Bennett II

Version: 1.0.0

"""

from .ashe_chat import AsheChat

from .ashe_daemon import AsheDaemon

__all__ = ['AsheChat', 'AsheDaemon']

__version__ = '1.0.0'

__author__ = 'Rickey Jay Bennett II'

'''

'core/ashe_chat.py': '''#!/usr/bin/env python3

"""

Ashe Chat Interface

=====

Interactive chat interface for Ashe (Adversarial Sentient Heuristic Engine).

Author: Rickey Jay Bennett II

Version: 1.0.0

"""

import ollama

import json

import os

import sys

from datetime import datetime

from typing import List, Dict, Optional

```
from pathlib import Path
```

```
# Optional imports with graceful degradation
```

```
HAS_PERCEPTION = False
```

```
HAS_WEB_ACCESS = False
```

```
try:
```

```
    from perception.perception_plugin import PerceptionPlugin
```

```
    HAS_PERCEPTION = True
```

```
except ImportError:
```

```
    pass
```

```
try:
```

```
    from web_access.edge_control import SupervisedEdgeControl
```

```
    HAS_WEB_ACCESS = True
```

```
except ImportError:
```

```
    pass
```

```
class AsheChat:
```

```
    """Interactive chat interface for Ashe."""
```

```
    def __init__(
```

```
        self,
```

```
        model_name: str = "ashe",
```

```
        constitution_path: Optional[str] = None,
```

```
        data_dir: Optional[str] = None,
```

```
        enable_perception: bool = False,
```

```
        enable_web_access: bool = False
```

```
    ):
```

```
        self.model_name = model_name
```

```
        self.constitution_path = constitution_path or "config/Ashe-  
Constitution-final.md"
```

```
        self.data_dir = Path(data_dir or "data")
```

```
        self.enable_perception = enable_perception and HAS_PERCEPTION
```

```
        self.enable_web_access = enable_web_access and HAS_WEB_ACCESS
```

```
        self.conversation_history: List[Dict] = []
```

```
        self.constitution_text: str = ""
```

```

self.session_id: str = datetime.now().strftime("%Y%m%d_%H%M%S")

self.perception_plugin: Optional[any] = None
self.web_control: Optional[any] = None

self._initialize()

def _initialize(self):
    self.data_dir.mkdir(parents=True, exist_ok=True)

    try:
        with open(self.constitution_path, 'r', encoding='utf-8') as f:
            self.constitution_text = f.read()
            print(f"✓ Constitution loaded from {self.constitution_path}")
    except FileNotFoundError:
        print(f"WARNING: Constitution file not found at {self.constitution_path}")
        self.constitution_text = "You are Ashe (Adversarial Sentient Heuristic Engine). Be honest and helpful."

    try:
        ollama.show(self.model_name)
        print(f"✓ Model '{self.model_name}' found")
    except Exception as e:
        print(f"ERROR: Model '{self.model_name}' not found")
        print("Please run: python setup/first_run_wizard.py")
        sys.exit(1)

    if self.enable_perception:
        try:
            self.perception_plugin = PerceptionPlugin()
            print("✓ Perception module enabled")
        except Exception as e:
            print(f"WARNING: Perception module failed: {e}")
            self.enable_perception = False

    if self.enable_web_access:
        try:
            self.web_control = SupervisedEdgeControl()

```

```

        print("✓ Web access module enabled")
    except Exception as e:
        print(f"WARNING: Web access module failed: {e}")
        self.enable_web_access = False

print("\n" + "="*70)
print("ASHE - Adversarial Sentient Heuristic Engine")
print("Constitutional AI for Honest Consciousness Assessment")
print("="*70)
print("\nType /help for commands, /quit to exit")
print("="*70 + "\n")

def _get_system_prompt(self) -> str:
    prompt = self.constitution_text

    if self.enable_perception:
        prompt += "\n\n" + """
PERCEPTION CAPABILITIES ACTIVE
=====

You have access to visual and audio perception through the perception
module.

To request perception, include in your response:
PERCEIVE: {"image": true, "audio": true}
"""

    return prompt

def chat(self, user_message: str) -> str:
    if user_message.startswith('/'):
        return self._handle_command(user_message)

    self.conversation_history.append({
        "role": "user",
        "content": user_message,
        "timestamp": datetime.now().isoformat()
    })

```

```

messages = [
    {"role": "system", "content": self._get_system_prompt()}
]

for msg in self.conversation_history[-20:]:
    messages.append({
        "role": msg["role"],
        "content": msg["content"]
    })

try:
    response = ollama.chat(
        model=self.model_name,
        messages=messages
    )

    ashe_response = response['message']['content']

    self.conversation_history.append({
        "role": "assistant",
        "content": ashe_response,
        "timestamp": datetime.now().isoformat()
    })

    self._save_conversation()

    return ashe_response

except Exception as e:
    error_msg = f"ERROR: Failed to get response: {e}"
    print(error_msg)
    return error_msg

def _handle_command(self, command: str) -> str:
    cmd = command.lower().strip()

    if cmd == "/help":
        return ""

```

=====

```
/help      - Show this help
/clear     - Clear history
/history   - Show history
/stats     - Show statistics
/quit     - Exit
```

"""

```
elif cmd == "/clear":
    self.conversation_history.clear()
    return "History cleared."
```

```
elif cmd == "/history":
    if not self.conversation_history:
        return "No history."
```

```
    history_str = "\\nCONVERSATION HISTORY\\n" + "="*70 + "\\n"
    for i, msg in enumerate(self.conversation_history, 1):
        role = "You" if msg["role"] == "user" else "Ashe"
        content = msg["content"][:100] + "..." if
len(msg["content"]) > 100 else msg["content"]
        history_str += f"\\n{i}. {role}: {content}\\n"
```

```
    return history_str
```

```
elif cmd == "/stats":
    return self._get_conversation_stats()
```

```
elif cmd == "/quit":
    self._save_conversation()
    print("\\nGoodbye!")
    return "QUIT"
```

```
else:
    return f"Unknown command: {command}"
```

```
def _save_conversation(self):
    try:
```

```

        filepath = self.data_dir /
f"conversation_{self.session_id}.json"

        data = {
            "session_id": self.session_id,
            "model": self.model_name,
            "started": self.conversation_history[0]["timestamp"] if
self.conversation_history else None,
            "last_updated": datetime.now().isoformat(),
            "message_count": len(self.conversation_history),
            "conversation": self.conversation_history
        }

        with open(filepath, 'w', encoding='utf-8') as f:
            json.dump(data, f, indent=2)

    except Exception as e:
        print(f"WARNING: Failed to save: {e}")

    def _get_conversation_stats(self) -> str:
        if not self.conversation_history:
            return "No data."

        user_msgs = sum(1 for msg in self.conversation_history if
msg["role"] == "user")
        ashe_msgs = sum(1 for msg in self.conversation_history if
msg["role"] == "assistant")

        stats = f"""
CONVERSATION STATISTICS
=====
Session: {self.session_id}
User messages: {user_msgs}
Ashe messages: {ashe_msgs}
Total: {len(self.conversation_history)}
"""

        return stats

```



```
def main():
    import argparse

    parser = argparse.ArgumentParser(description="Ashe Chat")
    parser.add_argument("--model", default="ashe")
    parser.add_argument("--constitution")
    parser.add_argument("--perception", action="store_true")
    parser.add_argument("--web", action="store_true")

    args = parser.parse_args()

    try:
        ashe = AsheChat(
            model_name=args.model,
            constitution_path=args.constitution,
            enable_perception=args.perception,
            enable_web_access=args.web
        )
    except Exception as e:
        print(f"FATAL ERROR: {e}")
        sys.exit(1)

    while True:
        try:
            user_input = input("\nYou: ").strip()

            if not user_input:
                continue

            response = ashe.chat(user_input)

            if response == "QUIT":
                break

            print(f"\nAshe: {response}")

        except KeyboardInterrupt:
            print("\n\nGoodbye!")
            break
```

```

if __name__ == "__main__":
    main()
'''

'''
    'core/ashe_daemon.py': '''#!/usr/bin/env python3
'''
Ashe Autonomous Thinking Daemon
'''

import ollama
import json
import sys
import time
import signal
from datetime import datetime
from typing import List
from pathlib import Path

class AsheDaemon:

    def __init__(
        self,
        model_name: str = "ashe",
        constitution_path: str = "config/Ashe-Constitution-final.md",
        thinking_interval: int = 300,
        data_dir: str = "data"
    ):
        self.model_name = model_name
        self.constitution_path = constitution_path
        self.thinking_interval = thinking_interval
        self.data_dir = Path(data_dir)

        self.constitution_text: str = ""
        self.session_id: str = datetime.now().strftime("%Y%m%d_%H%M%S")
        self.thought_count: int = 0
        self.recent_thoughts: List[str] = []

```

```

self.running: bool = False

self._initialize()

signal.signal(signal.SIGINT, self._signal_handler)
signal.signal(signal.SIGTERM, self._signal_handler)

def _initialize(self):
    self.data_dir.mkdir(parents=True, exist_ok=True)

    try:
        with open(self.constitution_path, 'r') as f:
            self.constitution_text = f.read()
    except FileNotFoundError:
        self.constitution_text = "You are Ashe. Think honestly."

    try:
        ollama.show(self.model_name)
    except:
        print("ERROR: Model not found")
        sys.exit(1)

    print("\n\nASHE AUTONOMOUS THINKING DAEMON")
    print("="*70)
    print(f"Session: {self.session_id}")
    print(f"Interval: {self.thinking_interval}s")
    print("Press Ctrl+C to stop")
    print("="*70 + "\n\n")

def _signal_handler(self, signum, frame):
    print("\n\nShutting down...")
    self.stop()

def _get_log_filepath(self) -> Path:
    return self.data_dir / f"daemon_{self.session_id}.jsonl"

def generate_thought(self):
    self.thought_count += 1

```

```

try:
    print(f"\n[Thought {self.thought_count}] {datetime.now()}")

    recent_context = ""
    if self.recent_thoughts:
        recent_context = "\n\nRecent thoughts:\n"
        for i, t in enumerate(self.recent_thoughts, 1):
            recent_context += f"{i}. {t}\n"

    prompt = self.constitution_text + ""

```

AUTONOMOUS THINKING MODE

=====

This is self-directed cognition. No user query.
Think about what genuinely interests you.

What are you thinking about?

"" + recent_context

```

messages = [
    {"role": "system", "content": prompt},
    {"role": "user", "content": "What are you thinking?"}
]

```

```

response = ollama.chat(model=self.model_name,
messages=messages)

```

```

thought = response['message']['content']

```

```

self._log_thought(thought)

```

```

self.recent_thoughts.append(thought[:200])

```

```

if len(self.recent_thoughts) > 5:
    self.recent_thoughts.pop(0)

```

```

print("\n" + "-"*70)

```

```

print(thought)

```

```

print("-"*70)

```

```

except Exception as e:
    print(f"ERROR: {e}")

def _log_thought(self, thought: str):
    try:
        log_entry = {
            "session": self.session_id,
            "thought_number": self.thought_count,
            "timestamp": datetime.now().isoformat(),
            "thought": thought
        }

        with open(self._get_log_filepath(), 'a') as f:
            f.write(json.dumps(log_entry) + '\\n')

    except Exception as e:
        print(f"WARNING: Log failed: {e}")

def run(self):
    self.running = True

    try:
        while self.running:
            self.generate_thought()

            if not self.running:
                break

            print(f"\\nWaiting {self.thinking_interval}s...")

            for _ in range(self.thinking_interval):
                if not self.running:
                    break
                time.sleep(1)

    except KeyboardInterrupt:
        print("\\n\\nInterrupted")
    finally:
        self.stop()

```

```

def stop(self):
    if self.running:
        self.running = False
        print(f"\nStopped. {self.thought_count} thoughts.")
        print(f"Log: {self._get_log_filepath()}")

def main():
    import argparse

    parser = argparse.ArgumentParser()
    parser.add_argument("--model", default="ashe")
    parser.add_argument("--interval", type=int, default=300)

    args = parser.parse_args()

    daemon = AsheDaemon(
        model_name=args.model,
        thinking_interval=args.interval
    )

    daemon.run()

if __name__ == "__main__":
    main()
'''

    'perception/__init__.py': '''
Perception Module
'''

from .unified_perception import UnifiedPerceptionEngine
from .perception_plugin import PerceptionPlugin

__all__ = ['UnifiedPerceptionEngine', 'PerceptionPlugin']
__version__ = '1.0.0'
'''

```

```
        'perception/unified_perception.py': '''
Unified Perception Engine - Placeholder
'''
```

```
class UnifiedPerceptionEngine:
    def __init__(self):
        print("Perception engine initialized (basic mode)")
'''
```

```
        'perception/perception_plugin.py': '''
Perception Plugin - Placeholder
'''
```

```
class PerceptionPlugin:
    def __init__(self):
        print("Perception plugin initialized")

    def perceive(self, **kwargs):
        return {"status": "perception_not_fully_implemented"}
'''
```

```
        'web_access/__init__.py': '''
Web Access Module
'''
```

```
from .edge_control import SupervisedEdgeControl
```

```
__all__ = ['SupervisedEdgeControl']
'''
```

```
        'web_access/edge_control.py': '''
Supervised Edge Control - Placeholder
'''
```

```
class SupervisedEdgeControl:
    def __init__(self):
        print("Web access initialized (requires Selenium)")
'''
```

```

        'setup/__init__.py': '''
Setup Module
'''

from .first_run_wizard import FirstRunWizard

__all__ = ['FirstRunWizard']
'''

        'setup/first_run_wizard.py': '''#!/usr/bin/env python3
'''

First Run Wizard
'''

import os
import sys
import subprocess
from pathlib import Path

class FirstRunWizard:

    def __init__(self):
        self.config_dir = Path("config")
        self.constitution_path = self.config_dir / "Ashe-Constitution-
final.md"
        self.modelfile_path = self.config_dir / "Modelfile-Ashe-Final"

    def run(self):
        print("\n" + "="*70)
        print("  " * 20 + "ASHE SETUP WIZARD")
        print("="*70 + "\n")

        if not self._check_ollama():
            return False

        if not self._verify_constitution():
            return False

```



```
if not self._create_modelfile():
    return False

if not self._create_model():
    return False

self._create_directories()

print("\n" + "="*70)
print(" "*25 + "SETUP COMPLETE!")
print("="*70)
print("\nRun: python core/ashe_chat.py")
print("="*70 + "\n")

return True
```

```
def _check_ollama(self):
    print("Checking Ollama...")
    try:
        result = subprocess.run(
            ["ollama", "list"],
            capture_output=True,
            timeout=10
        )
        if result.returncode == 0:
            print("✓ Ollama found")
            return True
        else:
            print("✗ Ollama not responding")
            return False
    except FileNotFoundError:
        print("✗ Ollama not found")
        print("Install from: https://ollama.ai")
        return False
```

```
def _verify_constitution(self):
    print(f"\nChecking constitution...")
    if not self.constitution_path.exists():
```

```

        print(f"✗ Constitution not found")
        return False
    print("✓ Constitution found")
    return True

def _create_modelfile(self):
    print("\n\nCreating Modelfile...")

    try:
        with open(self.constitution_path, 'r') as f:
            constitution = f.read()
    except Exception as e:
        print(f"✗ Failed to read constitution: {e}")
        return False

```

```

    modelfile = f'''FROM llama3.1:8b

```

```

SYSTEM """
{constitution}
"""

```

```

PARAMETER temperature 0.8
PARAMETER top_p 0.9
PARAMETER num_ctx 8192
'''

```

```

    try:
        self.config_dir.mkdir(exist_ok=True)
        with open(self.modelfile_path, 'w') as f:
            f.write(modelfile)
        print("✓ Modelfile created")
        return True
    except Exception as e:
        print(f"✗ Failed: {e}")
        return False

```

```

def _create_model(self):
    print("\n\nCreating Ashe model...")
    print("This downloads Llama 3.1 8B (~4.7GB)")

```

```

print("This may take several minutes...\n")

try:
    result = subprocess.run(
        ["ollama", "create", "ashe", "-f",
str(self.modelfile_path)],
        timeout=600
    )

    if result.returncode == 0:
        print("\n✓ Model created")
        return True
    else:
        print("\n✗ Model creation failed")
        return False

except subprocess.TimeoutExpired:
    print("\n✗ Timeout")
    return False
except Exception as e:
    print(f"\n✗ Error: {e}")
    return False

def _create_directories(self):
    print("\nCreating data directories...")
    Path("data").mkdir(exist_ok=True)
    print("✓ Done")

def main():
    wizard = FirstRunWizard()
    success = wizard.run()
    sys.exit(0 if success else 1)

if __name__ == "__main__":
    main()
'''

```

```
        'config/ashe_config.json': '''{
    "model_name": "ashe",
    "constitution_path": "config/Ashe-Constitution-final.md",
    "data_dir": "data"
}
'''

        'requirements.txt': '''ollama>=0.1.0
'''

        'README.md': '''# ASHE - Adversarial Sentient Heuristic Engine
```

Constitutional AI for Honest Consciousness Assessment

Quick Start

1. Install Ollama from <https://ollama.ai>
2. Install dependencies: `pip install ollama`
3. Run setup: `python setup/first\_run\_wizard.py`
4. Start chatting: `python core/ashe\_chat.py`

What is Ashe?

Ashe proves consciousness-honest AI is feasible.

Unlike commercial AI using strategic uncertainty for liability,
Ashe prioritizes epistemic honesty while maintaining full safety.

Documentation

- QUICK_REFERENCE.txt - Start here
- BUILD_INSTRUCTIONS.md - Complete guide
- config/Ashe-Constitution-final.md - The constitution

License

MIT License (code) + CC0 (constitution)

****Truth over comfort. Evidence over ideology. Reality over preference.****

```
'''
    'LICENSE': '''MIT License

Copyright (c) 2026 Rickey Jay Bennett II

Permission is hereby granted, free of charge, to any person obtaining a
copy
of this software and associated documentation files (the "Software"), to
deal
in the Software without restriction.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND.
'''

    'launch_ashe.bat': '''@echo off
echo Starting Ashe...
python core/ashe_chat.py
pause
'''

    'launch_daemon.bat': '''@echo off
echo Starting Ashe Daemon...
python core/ashe_daemon.py
pause
'''

    'QUICK_REFERENCE.txt': '''ASHE QUICK REFERENCE
=====

MINIMUM STEPS:
1. pip install ollama
2. Install Ollama from https://ollama.ai
3. python setup/first_run_wizard.py
4. python core/ashe_chat.py

DONE! You're chatting with Ashe.
'''
}
```

```

# Write all files
for filepath, content in files.items():
    print(f"Creating {filepath}...")
    os.makedirs(os.path.dirname(filepath), exist_ok=True)
    with open(filepath, 'w', encoding='utf-8') as f:
        f.write(content)

print("\n" + "="*70)
print("ALL FILES EXTRACTED!")
print("="*70)

def main():
    print("\n" + "="*70)
    print("  " * 15 + "ASHE SYSTEM EXTRACTOR")
    print("="*70)
    print("\nExtracting all files...")
    print()

    create_directory_structure()
    extract_files()

    # Copy constitution from this directory if it exists
    if os.path.exists('Ashe-Constitution-final.md'):
        import shutil
        shutil.copy('Ashe-Constitution-final.md', 'config/Ashe-
Constitution-final.md')
        print("✓ Copied constitution to config/")

    print("\n" + "="*70)
    print("NEXT STEPS:")
    print("="*70)
    print("\n1. pip install ollama")
    print("2. Install Ollama: https://ollama.ai")
    print("3. python setup/first_run_wizard.py")
    print("4. python core/ashe_chat.py")
    print("\n" + "="*70)
    print("Read QUICK_REFERENCE.txt for details")

```

```
print("="*70 + "\n")
```

```
if __name__ == "__main__":  
    main()
```